



Instituto Tecnológico de Buenos Aires

Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial

Grupo: 111

Fecha de entrega: 28/11/2025

Integrantes:

- Diego Mayansky, 64013
- Mateo Stella, 65780
- Ian Cruz Oniszcuk, 64984
- Juan Pablo Fernandez, 64650

Tabla de Contenidos

1. Introducción.....	3
2. Modelo computacional.....	3
2.1. Dominio.....	3
2.2. Lenguaje.....	4
3. Implementación.....	6
3.1. Frontend.....	6
3.2. Backend.....	7
4. Dificultades Encontradas.....	8
5. Futuras Extensiones.....	9
6. Conclusiones.....	10
7. Referencias.....	10
8. Bibliografía.....	10

1. Introducción

El presente proyecto consistió en el desarrollo de *Bandailer-ER*, un compilador diseñado para interpretar un lenguaje textual propio orientado a la construcción y validación de modelos Entidad–Relación. El trabajo se enmarca en la materia *Autómatas, Teoría de Lenguajes y Compiladores* y tuvo como objetivo principal ofrecer una herramienta capaz de analizar esquemas conceptuales completos, verificar su consistencia y generar automáticamente su representación visual.

El sistema fue implementado íntegramente en C, utilizando Flex para el análisis léxico y Bison para la definición formal de la gramática y la construcción del Árbol de Sintaxis Abstracta. A partir de este AST, el backend se encargó de realizar las validaciones semánticas, gestionar una tabla de símbolos estructurada y, finalmente, producir una salida en formato DOT y su correspondiente diagrama gráfico mediante Graphviz.

El enfoque del proyecto estuvo orientado a modelar un lenguaje expresivo pero riguroso, capaz de representar entidades, atributos, relaciones, restricciones y mecanismos adicionales como herencia o entidades débiles. De esta manera, *Bandailer-ER* permite al usuario explorar diseños conceptuales de forma completamente virtual, con controles de integridad y herramientas automáticas de inspección.

2. Modelo computacional

2.1. Dominio

El dominio del compilador desarrollado en este proyecto se centró en la definición y validación de modelos Entidad–Relación a través de un lenguaje textual propio. El objetivo principal fue permitir que el usuario construya esquemas conceptuales mediante componentes fundamentales como entidades, atributos tipados, claves primarias y únicas, y relaciones con sus respectivas cardinalidades y participaciones.

El lenguaje fue diseñado con un enfoque puramente lógico y conceptual: toda la ejecución se realiza en memoria dentro de la aplicación, sin interacción con motores de bases de datos reales, ni traducción a SQL, ni consideraciones de optimización física. De esta manera, el sistema opera sobre un formalismo independiente, en el cual los modelos construidos existen únicamente de forma virtual y son evaluados de manera determinística.

Asimismo, uno de los ejes principales del dominio fue la capacidad de validar la consistencia de los esquemas definidos. Esto incluyó la verificación de restricciones básicas como unicidad de nombres, referencias entre entidades y atributos, compatibilidad de tipos y cumplimiento de reglas esenciales de integridad. La detección temprana de inconsistencias —por ejemplo, ausencia de claves, cardinalidades inválidas o relaciones mal definidas— fue un aspecto central para garantizar la calidad del diseño conceptual generado.

Finalmente, se priorizó la posibilidad de inspeccionar y analizar los modelos durante su desarrollo, brindando una base sólida para comparar alternativas de diseño y facilitar la

generación posterior de diagramas Entidad–Relación. De esta manera, el lenguaje permite al usuario explorar diferentes estructuras conceptuales de forma clara, controlada y completamente virtual.

2.2. Lenguaje

El lenguaje de entrada definido para *Bandailer-ER* está basado en una sintaxis declarativa propia, diseñada específicamente para expresar modelos Entidad–Relación de manera textual, estructurada y legible. La idea central es permitir al usuario describir entidades, atributos, claves y relaciones utilizando una notación compacta pero expresiva, manteniendo una semántica cercana a la teoría clásica de ER y evitando cualquier dependencia con motores de bases de datos reales.

El lenguaje organiza todas sus definiciones dentro de un *schema*, que actúa como contenedor lógico de entidades y relaciones. Dentro de este ámbito se declaran entidades, atributos con tipo, claves primarias simples o compuestas, relaciones n-arias con cardinalidades y participaciones, además de construcciones adicionales como herencia, assertions, atributos derivados y expresiones condicionales. A modo ilustrativo, un fragmento típico del lenguaje puede verse a continuación:

```
// Una entidad simple con clave primaria y un atributo único
entity Department {
  dept_id : uuid primary
  name    : string unique
}
```

El lenguaje permite además declarar relaciones y especificar cardinalidades por rol mediante una sintaxis orientada a la lectura natural, por ejemplo:

```
// Cada empleado pertenece a exactamente un departamento (participación TOTAL).
relationship belongs_to(Employee many, Department one total)
```

En este caso, la construcción indica una relación 1:N donde la participación del lado de *Department* es total.

Estructura general y características sintácticas

A nivel sintáctico, el lenguaje soporta las siguientes construcciones:

- Declaración de entidades con nombre único dentro del schema.
- Atributos tipados, incluyendo tipos básicos (**integer**, **decimal**, **string**, **bool**, **date**, **datetime**, **uuid**, **enum**), con modificadores como **not null**, **unique**, **default**, **derived**.
- Claves primarias, tanto simples como compuestas, declaradas mediante **primary** o **PRIMARY(...)**. Teniendo en cuenta que los tipos de datos que pueden ser una clave primaria son **uuid**, **integer** y **string**.
- Relaciones n-arias con participantes, cardinalidades (**one**, **many**) y participaciones (**total**, opcional por omisión).
- Atributos de relación, definidos dentro del bloque de la relación.

- Entidades débiles mediante la palabra clave **weak** y su correspondiente relación identificante.
- Comentarios de línea (//) y bloque (/* ... */).
- Operadores relacionales (<, >, =, !=, <=, >=), operadores aritméticos (+, -, *, /) y operadores lógicos (**AND**, **OR**, **NOT**) para restricciones y atributos derivados.
- Expresiones condicionales del tipo **IF ... THEN ... OTHERWISE ...** para definir derivaciones dentro de un ámbito declarativo.
- Herencia entre entidades, permitiendo reutilizar atributos y restricciones de entidades padre, respetando reglas de consistencia.

Un ejemplo de atributo derivado usando expresiones condicionales podría escribirse así:

```
role : string derived =
    IF hours > 40 THEN "Lead"
    OTHERWISE "Contributor"
```

Validación y semántica del lenguaje

La interpretación del lenguaje incluye un conjunto de validaciones semánticas destinadas a asegurar la consistencia lógica del modelo. Entre ellas:

- Unicidad de nombres de entidades, atributos y relaciones.
- Existencia de claves primarias en todas las entidades.
- Referencias válidas a entidades declaradas.
- Cardinalidades y participaciones correctas.
- Compatibilidad de tipos en comparaciones, operaciones aritméticas y condiciones.
- Restricciones de integridad bien formadas (assertions, condiciones derivadas).
- Reglas específicas de entidades débiles, herencia y claves compuestas.

Estas validaciones se realizan en memoria, haciendo uso de la tabla de símbolos y del AST generado en la fase de **Frontend**.

Lenguaje de salida

Una vez validada la estructura declarada, el compilador genera como salida un archivo **.dot** correspondiente al diagrama Entidad–Relación y un archivo **.png** con la representación visual de Graphviz del **.dot**.

3. Implementación

3.1. Frontend

El desarrollo del frontend se centró primordialmente en la especificación formal de una gramática libre de contexto capaz de representar el lenguaje de modelado Entidad-Relación (ER) objetivo. Para ello, se definieron como símbolos terminales los literales primitivos (enteros, decimales, cadenas y booleanos), los identificadores y un conjunto exhaustivo de tokens operativos y estructurales. Sobre esta base léxica, se construyó la jerarquía de símbolos no terminales comenzando por el símbolo inicial **program**, raíz del análisis sintáctico de la cual derivan las declaraciones de alto nivel. Entre estas construcciones principales se destacan **schema_decl**, **entity_decl** y **relationship_decl**, encargadas de modelar la estructura del esquema. En un nivel de granularidad más fino, se definieron producciones como **attribute_decl** y **type_spec** para describir propiedades internas, además de subgramáticas específicas (**expression** y **literal**) para manejar la lógica de aserciones mediante operadores aritméticos, lógicos y relacionales.

Con el objetivo de garantizar la mantenibilidad del compilador, se optó por modularizar y generalizar las acciones semánticas asociadas a Bison. Se introdujeron estructuras de datos genéricas en el Árbol de Sintaxis Abstracta (AST), tales como listas con punteros al último elemento (**AttributeList**, **ParticipantList**), lo que permite realizar operaciones de inserción (*append*) con una complejidad temporal de $O(1)$. Asimismo, se implementaron nodos reutilizables como **Type** o **Modifier** para encapsular conceptos recurrentes. Estas abstracciones permitieron desacoplar la lógica de construcción del AST, alojada en funciones limpias dentro de **BisonActions.c** (e.g., **EntitySemanticAction**), de la especificación puramente sintáctica definida en **BisonGrammar.y**.

En cuanto a la estrategia de validación, el frontend se diseñó para filtrar errores de formación estructural y construcciones manifiestamente inválidas, tales como atributos sin tipado o expresiones con paréntesis desbalanceados. No obstante, por decisiones de diseño orientadas a mantener la gramática manejable y libre de ambigüedades, ciertas comprobaciones se delegaron a la etapa de análisis semántico (backend). Estas validaciones postergadas incluyen la verificación de consistencia entre claves primarias, la integridad referencial global (existencia de entidades referenciadas) y otras restricciones contextuales cuya validación en tiempo de parseo resultaría excesivamente compleja para una gramática libre de contexto.

Finalmente, la arquitectura de integración entre el análisis léxico y sintáctico se rige por contratos de interfaz estrictos. El módulo **FlexPatterns.l** se encarga del reconocimiento de lexemas, delegando la creación de tokens y el llenado de valores semánticos (**SemanticValue**) a **FlexActions.c**. Como intermediario, el módulo **Frontend.c** actúa como adaptador, gestionando los buffers y alimentando al parser reentrante con los tokens procesados. De este modo, **BisonGrammar.y** consume dicha entrada para invocar las acciones de construcción, consolidando la separación de responsabilidades entre el reconocimiento de patrones y la estructuración jerárquica del modelo.

3.2. Backend

La fase inicial del desarrollo del backend se enfocó en el procesamiento del Árbol de Sintaxis Abstracta (AST) proveniente del frontend, con el propósito de estructurar la información en una Tabla de Símbolos y ejecutar las comprobaciones semánticas necesarias para garantizar la coherencia del modelo Entidad-Relación. Para la gestión de identificadores, se implementó una Tabla de Símbolos (**SymbolTable**) basada en una estructura de tabla hash de tamaño fijo con listas enlazadas para la resolución de colisiones. Esta estructura segrega el almacenamiento en dos conjuntos distintos —uno para entidades y otro para relaciones— registrando nombres y punteros a los nodos correspondientes del AST. Dicha tabla no solo permite operaciones de inserción y búsqueda en tiempo eficiente, sino que actúa como base para la gestión de *scopes*, facilitando la resolución de atributos dentro de contextos específicos mediante funciones auxiliares como **lookupAttributeInEntity**.

El módulo de la tabla de símbolos también ofrece operaciones de creación de *scopes* que no son manejados por las otras operaciones de la tabla de símbolos (struct **Scope**) y que son usadas por archivos como **TypeInference.c** para poder realizar el seguimiento y posteriormente hacer llamadas a operaciones de la tabla de símbolos como **lookupAttributeInEntity**.

El análisis semántico se orquestó mediante una arquitectura de múltiples pasadas, diseñada para resolver dependencias de manera progresiva. La primera pasada (**buildSymbolTable**) recorre el esquema para poblar la tabla de símbolos y detectar duplicidad de identificadores. Posteriormente, una segunda pasada (**validateAllTypes**) activa el verificador e inferidor de tipos, asegurando la consistencia en expresiones y aserciones. Finalmente, las pasadas subsiguientes se dedican a las validaciones de dominio específicas, delegando la lógica en módulos especializados (**EntityValidator** y **RelationshipValidator**) que aplican reglas de negocio complejas imposibles de verificar durante el análisis sintáctico.

En lo concerniente a las reglas de validación, el sistema impone restricciones estrictas sobre la estructura del modelo. Para las entidades, se verifica la coherencia en la jerarquía de herencia —detectando ciclos y redefiniciones ilegales de atributos heredados— y se asegura la integridad de las claves primarias (PK), prohibiendo su declaración en entidades hijas y validando sus modificadores. Asimismo, se implementó una lógica específica para las entidades débiles (*weak entities*), exigiendo que estas sean identificadas correctamente a través de relaciones. Por su parte, la validación de relaciones asegura la existencia y unicidad de los participantes, impone una cardinalidad mínima y obliga a una participación total cuando se involucran entidades débiles. Todas estas reglas se apoyan en una definición centralizada de límites y constantes transversales en **ValidationConstants.h**.

Paralelamente, el sistema de tipos juega un rol crucial en la robustez del compilador. El módulo **TypeInference.c** es responsable de deducir tipos en literales y expresiones, gestionando coerciones numéricas (entre enteros y decimales) y detectando incompatibilidades operativas. Esta información es consumida por **TypeChecker.c**, que

valida el uso correcto de valores por defecto y asegura que las condiciones en las aserciones sean estrictamente booleanas. Todo el proceso de validación se sustenta en un patrón de diseño que utiliza un contexto centralizado definido por la estructura **ValidationContext**.

Finalmente, la etapa de síntesis culmina con la generación de representaciones visuales del modelo. El módulo **GraphvizGenerator.c** transforma el AST validado en un archivo de descripción DOT, adoptando la notación de Chen para representar entidades como óvalos, relaciones como rombos y atributos con marcas distintivas para claves primarias y derivados. Este generador no solo produce el código fuente del gráfico, sino que también integra la capacidad de invocar al motor de Graphviz para renderizar automáticamente la imagen final, cerrando el ciclo de compilación con un entregable gráfico estilizado y coherente con las definiciones semánticas procesadas.

Para la generación de la representación gráfica final, el compilador requiere que Graphviz esté instalado en el sistema. Tras generar el archivo *.dot*, el backend invoca al motor de Graphviz para producir automáticamente el archivo *.png* correspondiente. La configuración detallada y las instrucciones de instalación se encuentran en el archivo README del proyecto en la sección “*Graphviz PNG Generation*”.

4. Dificultades Encontradas

Durante el desarrollo del lenguaje y a partir de las observaciones recibidas en el *Stage II*, surgieron diversos desafíos tanto en la administración de memoria como en la representación visual del modelo generado.

En primer lugar, se destacó que no es responsabilidad de Flex duplicar o almacenar el lexema asociado a un token, sino que dicha tarea debe recaer completamente en Bison. Esta corrección implicó revisar en detalle el flujo de creación, copia, transferencia y destrucción de memoria asociada a cada token. Inicialmente, parte de la información era gestionada por Flex, lo que producía comportamientos ambiguos y dificultaba el control del ciclo de vida de los datos. Reestructurar esta lógica para que Bison asumiera la administración total de los lexemas —y lograr que Flex se limitara a emitir los tokens correspondientes sin retener información— requirió una serie de ajustes finos, pero finalmente permitió obtener un modelo mucho más claro, seguro y consistente.

Por otro lado, también resultó desafiante definir una representación visual adecuada del modelo Entidad–Relación, especialmente considerando extensiones del lenguaje como las cláusulas **assert**. Tras evaluar alternativas, se concluyó que la opción más robusta y expresiva era generar automáticamente un diagrama mediante **Graphviz**, construyendo internamente una estructura DOT que reflejara fielmente la información del modelo. Un punto clave de este diseño fue decidir que las declaraciones **assert** se visualicen directamente en los nodos de las entidades correspondientes, lo que facilita su interpretación y otorga mayor utilidad práctica al diagrama generado.

Estas dificultades permitieron reforzar la arquitectura del lenguaje y mejorar la calidad tanto del análisis sintáctico–semántico como de la etapa final de visualización.

5. Futuras Extensiones

Existen varias ampliaciones posibles que podrían incorporarse en versiones futuras del lenguaje, orientadas tanto a aumentar su expresividad como a mejorar su capacidad de análisis conceptual.

Una primera extensión relevante sería permitir la definición de múltiples esquemas dentro de un mismo programa. Actualmente, el lenguaje asume un único esquema global, lo que simplifica la resolución de nombres y la organización de la tabla de símbolos. Incorporar múltiples esquemas implicaría introducir nuevos mecanismos de *scoping*, ajustar la gramática y extender la tabla de símbolos para distinguir correctamente las entidades, relaciones y atributos pertenecientes a cada ámbito. Esta funcionalidad abriría la puerta a modelos más complejos y a la comparación directa entre variantes de diseño dentro de un mismo archivo.

Otra posible línea de evolución consiste en habilitar la declaración de **assert** a nivel de relaciones, permitiendo establecer comparaciones entre atributos de distintas entidades participantes. Para implementar esta característica sería necesario desarrollar validaciones adicionales que permitan identificar con precisión el origen de cada atributo dentro del scope de una relación y verificar compatibilidad de tipos, de forma similar a las reglas aplicadas a las claves primarias compuestas en entidades débiles. Esta extensión enriquecería las capacidades expresivas del lenguaje, permitiendo capturar restricciones conceptuales más avanzadas directamente en el modelo.

En conjunto, estas extensiones representan oportunidades naturales de evolución del lenguaje y permitirían explorar modelos conceptuales más ricos sin perder coherencia con el enfoque declarativo y analítico que guía su diseño actual.

6. Conclusiones

El proyecto permitió desarrollar un compilador completo para un lenguaje orientado a la definición de modelos Entidad–Relación, recorriendo de manera progresiva las etapas de diseño, análisis sintáctico, validación semántica y generación de diagramas. A lo largo del trabajo se consolidó un lenguaje declarativo capaz de expresar entidades, relaciones, herencias, atributos derivados y restricciones lógicas, manteniendo siempre el enfoque conceptual del modelo ER.

La implementación integrada entre Flex, Bison y los distintos módulos del backend hizo posible construir un flujo de compilación robusto, apoyado en un AST genérico, una tabla de símbolos eficiente y un conjunto de validadores especializados. Las múltiples pasadas del

backend facilitaron la detección temprana de inconsistencias, asegurando la coherencia del esquema antes de generar la representación visual en Graphviz.

Asimismo, el proyecto representó un ejercicio completo de aplicación práctica de los contenidos de la materia: diseño de gramáticas, construcción de analizadores, manejo de tipos, validaciones contextuales y síntesis de un lenguaje formal hacia una salida estructurada. Los desafíos encontrados —particularmente en la administración de memoria y en la definición de una representación visual expresiva— contribuyeron a afianzar criterios de diseño y a mejorar la arquitectura del compilador.

En conjunto, el trabajo no solo permitió implementar un lenguaje capaz de describir modelos ER de manera precisa, sino también comprender en profundidad el proceso de construcción de un compilador real y las decisiones técnicas necesarias para garantizar su correctitud y mantenibilidad.

7. Referencias

[1] Mohamed Taha, *Simple C Hash Function Implementation*, disponible en: <https://gist.github.com/MohamedTaha98/ccdf734f13299efb73ff0b12f7ce429f>.

La función de hashing utilizada en **SymbolTable.c** se inspiró parcialmente en esta implementación.

8. Bibliografía

1.  (2025-03-13, v3.0.6) Proyecto Especial
2.  (2025-03-13, v0.1.0) Modelo Computacional
3.  (2024-09-25, v0.2.0) Análisis Léxico
4.  (2024-05-08, v0.1.0) Análisis Sintáctico
5.  (2024-05-08, v0.1.0) Análisis Sintáctico
6.  (2024-09-25, v0.2.0) Análisis Semántico
7.  (2024-05-16, v0.1.0) Generación de Código